

Enrico Bravi

enrico.bravi@polito.it

Module E – OS Security

E1 – OS Introduction



**Politecnico
di Torino**

Department of Control and
Computer Engineering



TORSEC

Computer and Network Security Group



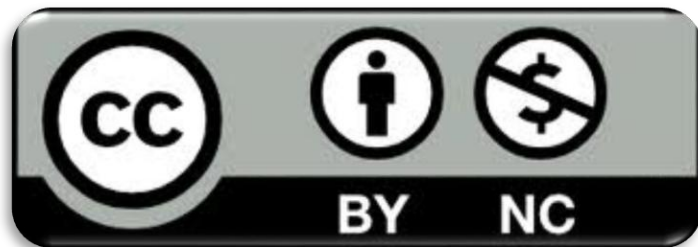
Acknowledgments

This lecture deck *adapts* and *reorganizes* selected teaching material originally developed by Nicolò Maunero (CINI / Cybersecurity National Lab). The presentation includes material from different sources (OSTEP, MIT, FreeRTOS, Usha Barad, Linux Manpage) including the material provided by Silberschatz, Galvin and Gagne, whose valuable help is here acknowledged and highly appreciated. Portions of the content are reused under the Creative Commons BY-NC license (see license slide). All modifications, additions, re-organization, and any errors are the responsibility of the course instructor.

License & Disclaimer

License Information

This presentation is licensed under the
Creative Commons BY-NC License



To view a copy of the license, visit:

<http://creativecommons.org/licenses/by-nc/3.0/legalcode>

Disclaimer

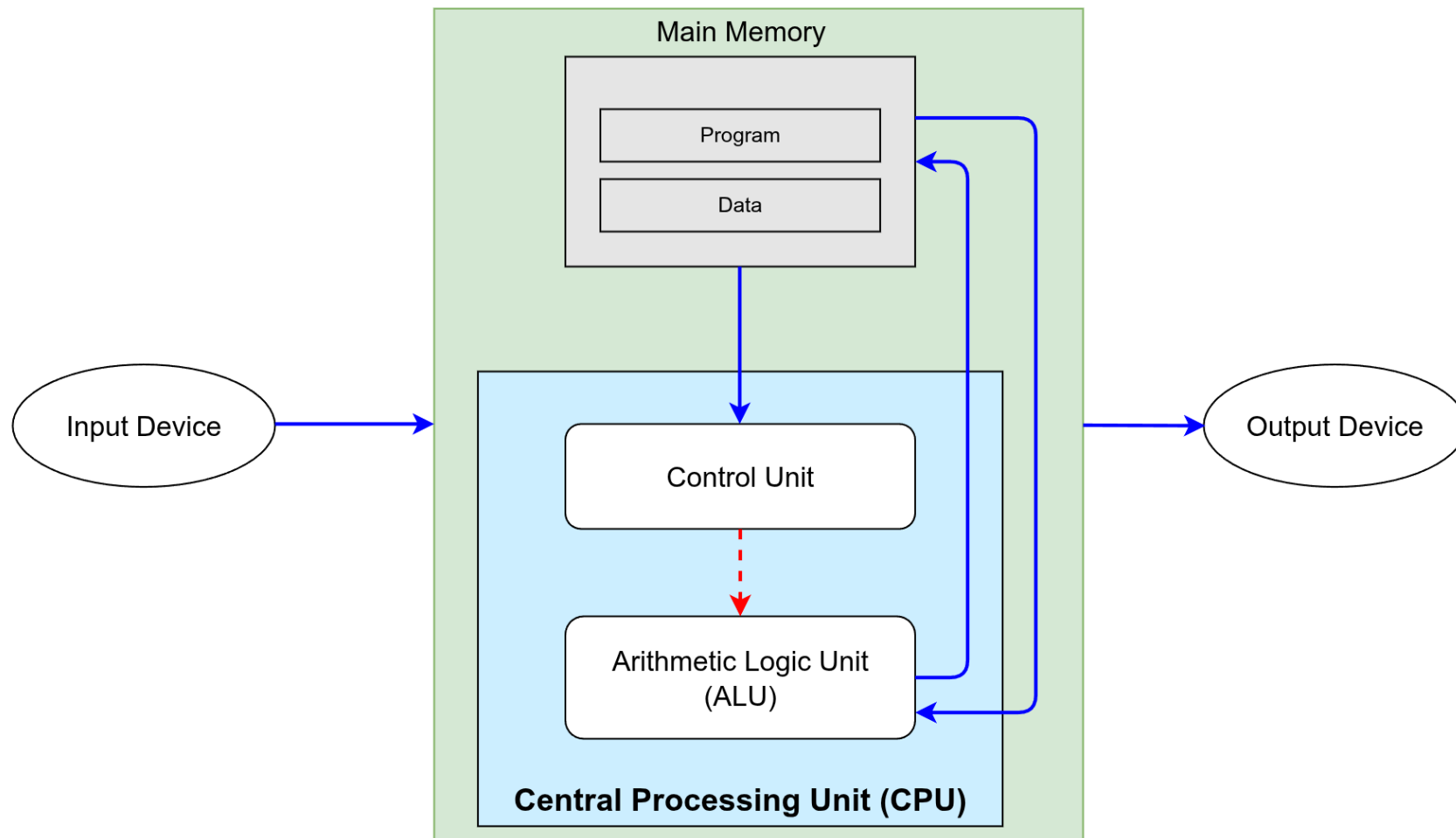
- We disclaim any warranties or representations as to the accuracy or completeness of this material.
- Materials are provided “as is” without warranty of any kind, either express or implied, including without limitation, warranties of merchantability, fitness for a particular purpose, and non-infringement.
- Under no circumstances shall we be liable for any loss, damage, liability or expense incurred or suffered which is claimed to have resulted from use of this material.
- This material is used and adapted within the course “Cybersecurity and National Defense”.



Introduction

- what is an operating system (OS)?
- an organized collection of software extensions of hardware that serve as:
 - control routines for operating a computer (for example, to gain access to computer resources (like file I/O))
 - an environment for execution of programs
 - an OS is an interface between the end user and the hardware

von Neumann Architecture





Software Execution (I)

- instructions reside in main memory (RAM)
- the CPU fetches each instruction and execute it
- instructions (aka software) need to be moved from persistent storage (e.g., HDD, SSD) to main memory
- on simple devices that execute only one application (e.g., IoT), only the executed application needs to be loaded in RAM
- when the device starts, a special component called **bootloader** moves the application in RAM where the CPU will be able to fetch the instructions and execute them

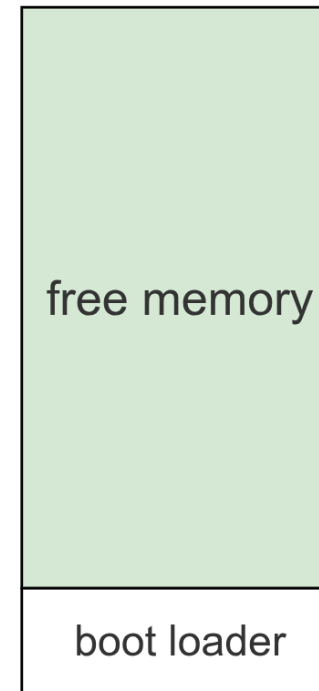


Software Execution (II)

- on general purpose machines, the OS substitute the bootloader for user applications
- the OS runs at a higher privileged level, in order to load applications in RAM and allocate the main memory when requested
- this hides several specific hardware aspects to developers and users

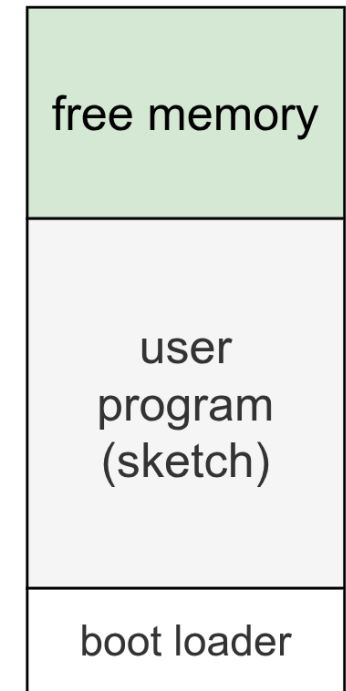
Example - Arduino

- running a single task
- NO operating system running
- single memory space



(a)

System startup



(b)

Program running



Example - FreeBSD

- full OS (UNIX variant)
- multi-tasking execution
- multi-user
- process lifecycle management

high
memory

kernel

free memory

process C

interpreter

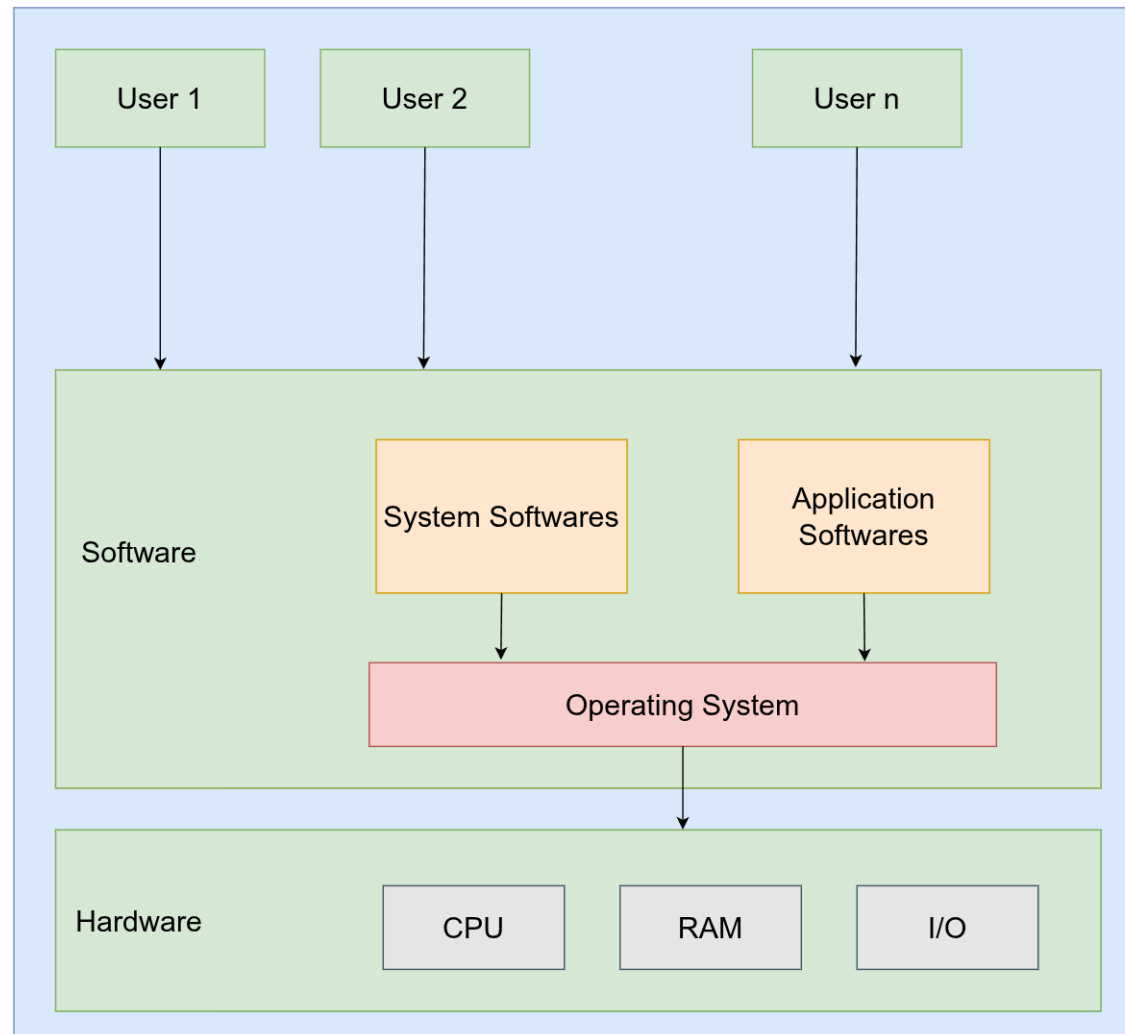
process B

low
memory

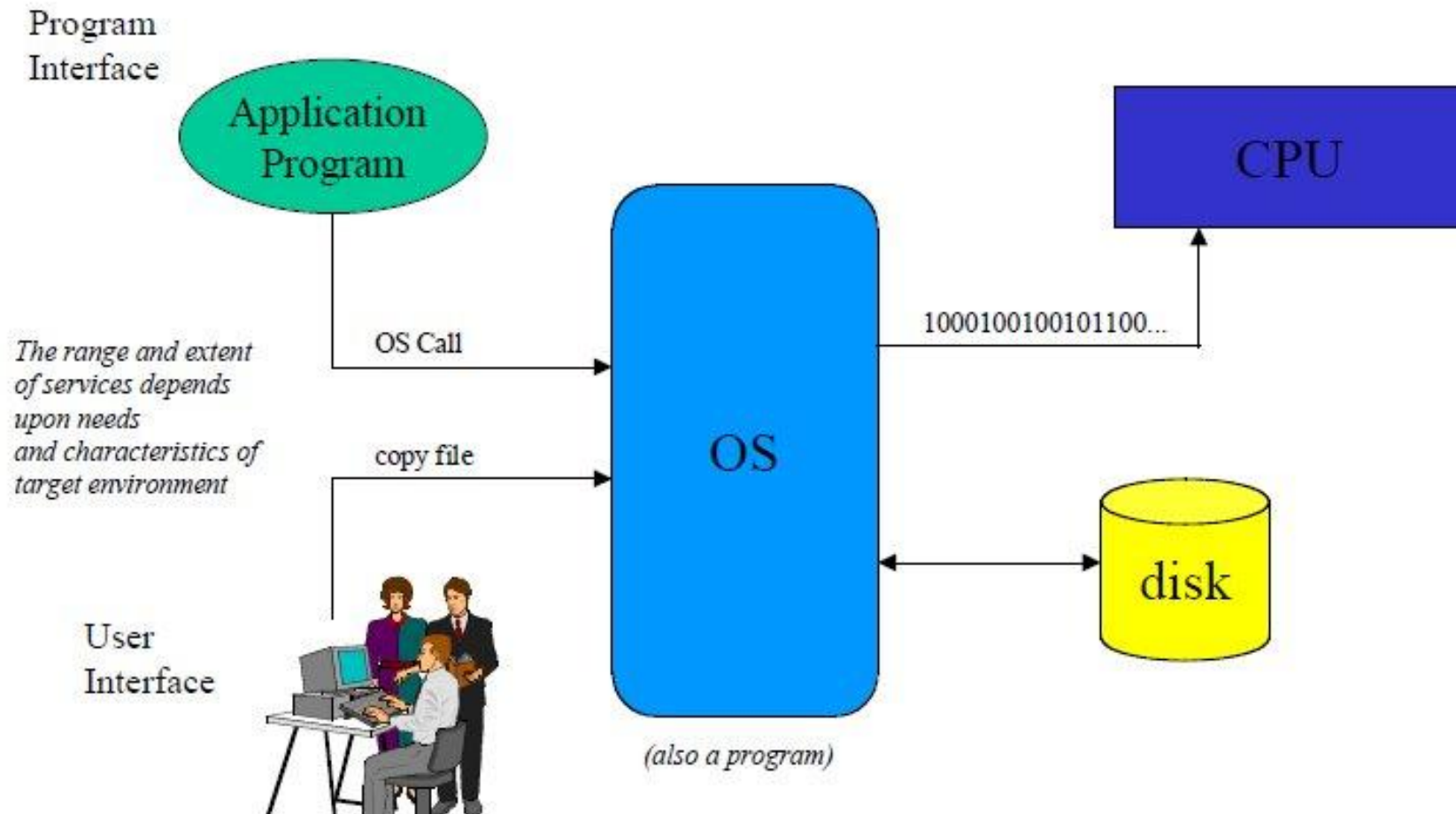
process D



OS – Layer of Abstraction



OS – General Purpose Computer





Operating Systems Purposes

- manages the memory
- manages the processes (services and applications)
- manages the connected devices (external and internal)
- manages files
- **security**
- ...



Operating System Services (I)

- the OS provide an execution environment for executing programs and services
- the main services provided are:
 - user interface: it can be based on Command-Line (CLI), Graphics User Interface (GUI)
 - program execution: the OS load a program in the main memory and run it, handling errors during execution
 - I/O operations: it manages the input/output from storage devices
 - filesystem manipulation: it manages the creation and manipulation of files, in particular managing permissions



Operating System Services (II)

- communications: the OS manages the communication between the several programs running
- error detection: it needs to be aware of possible errors during execution and handle them
- resource allocation: the OS needs to manage multi user/program scenario, where different entities request system resources
- logging: keeping track of the event happened on the system

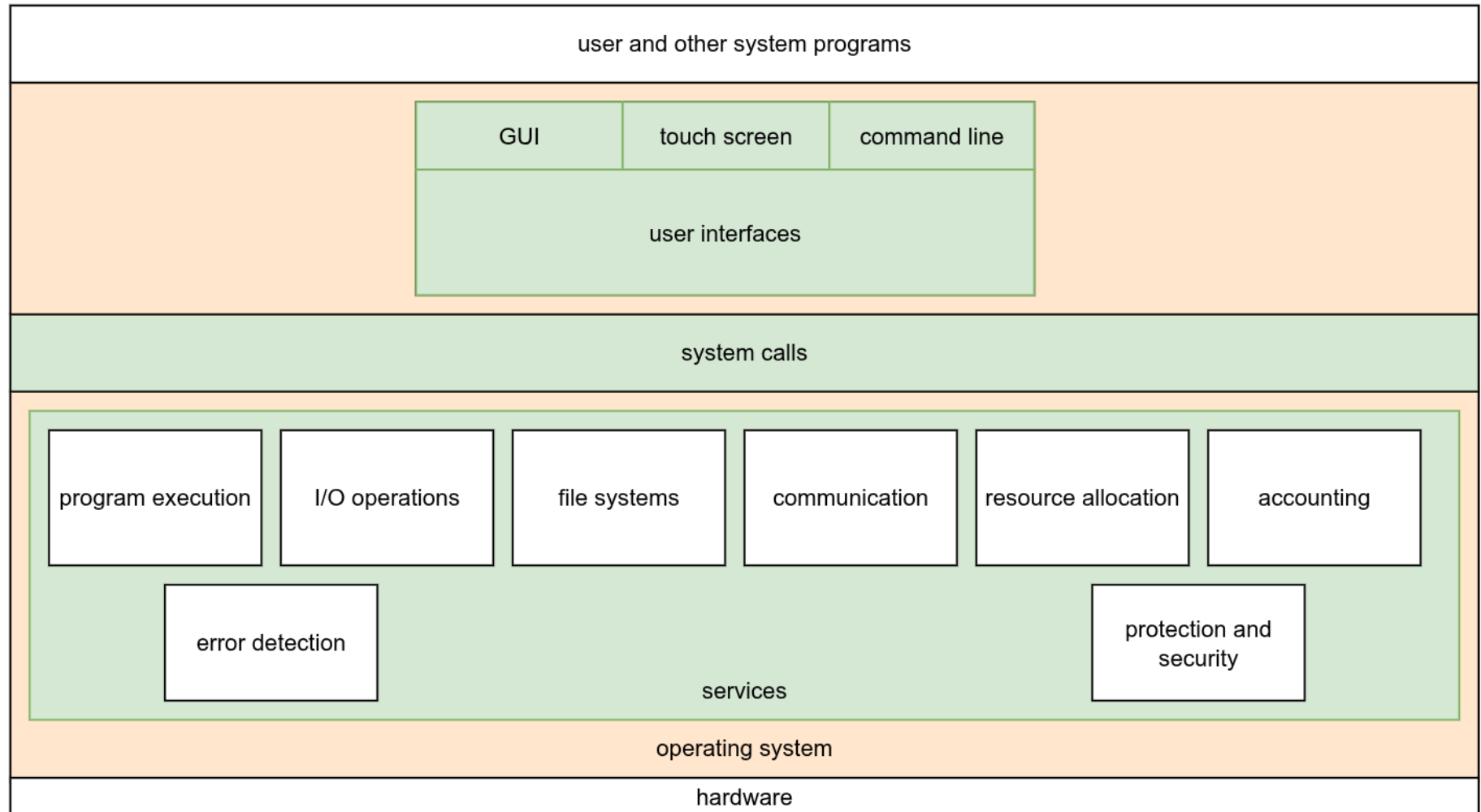


Operating System Services (III)

- protection and security:
 - protection: the access to all resources of the system is controlled
 - security: it concerns access attempts from outsiders of the system (user authentication)



OS General View





Command Line Interface

- the CLI acts as the first interface with the system permitting to directly send commands
- it can be implemented in the core of the OS (kernel) or as a system program
- it fetches commands from the user (program names, built-in) and execute them



Shell Command Interpreter

```
lun 13 apr 2026, 11:07:44, CEST

[~]$ lsusb
Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 003 Device 003: ID 27c6:63ac Shenzhen Goodix Technology Co.,Ltd. Goodix USB2.0 MISC
Bus 003 Device 002: ID 0c45:6a22 Microdia Integrated Webcam HD
Bus 003 Device 011: ID 1224:2a25 Jieli Technology USB PHY 2.0
Bus 003 Device 009: ID 320f:50c3 Telink 2.4G Receiver
Bus 003 Device 012: ID 0bda:1100 Realtek Semiconductor Corp. HID Device
Bus 003 Device 013: ID 17ef:a048 Lenovo Lenovo MS30 Soundbar
Bus 003 Device 010: ID 1a40:0801 Terminus Technology Inc. USB 2.0 Hub
Bus 003 Device 008: ID 0bda:5411 Realtek Semiconductor Corp. RTS5411 Hub
Bus 003 Device 007: ID 2109:2813 VIA Labs, Inc. VL813 Hub
Bus 003 Device 006: ID 05e3:0610 Genesys Logic, Inc. Hub
Bus 003 Device 014: ID 001f:0b21 Generic USB Audio
Bus 003 Device 005: ID 05e3:0610 Genesys Logic, Inc. Hub
Bus 003 Device 004: ID 8087:0033 Intel Corp.
Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 004: ID 0bda:8153 Realtek Semiconductor Corp. RTL8153 Gigabit Ethernet Adapter
Bus 002 Device 003: ID 05e3:0626 Genesys Logic, Inc. USB3.1 Hub
Bus 002 Device 002: ID 05e3:0625 Genesys Logic, Inc. USB3.2 Hub
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
[14ms][~]$ uname -a
Linux XPS-15-9530 6.8.0-107-generic #107~22.04.1-Ubuntu SMP PREEMPT_DYNAMIC Wed Mar 18 23:40:43 UTC x86_64 x86_64 x86_64 GNU/Linux
[30ms][~]$ free -g
              total        used        free      shared  buff/cache   available
Mem:           31          21           0           0           8           8
Swap:           9           0           9
[26ms][~]$
```



Graphical User Interface (GUI)

- It's a more user-friendly interface
 - it exploits mouse, keyboard and monitor
 - it uses icons, representing files, programs, actions, etc.
 - more intuitive actions for common operations (program execution, file creation, etc.)
- main commercial OS include GUI



System Calls

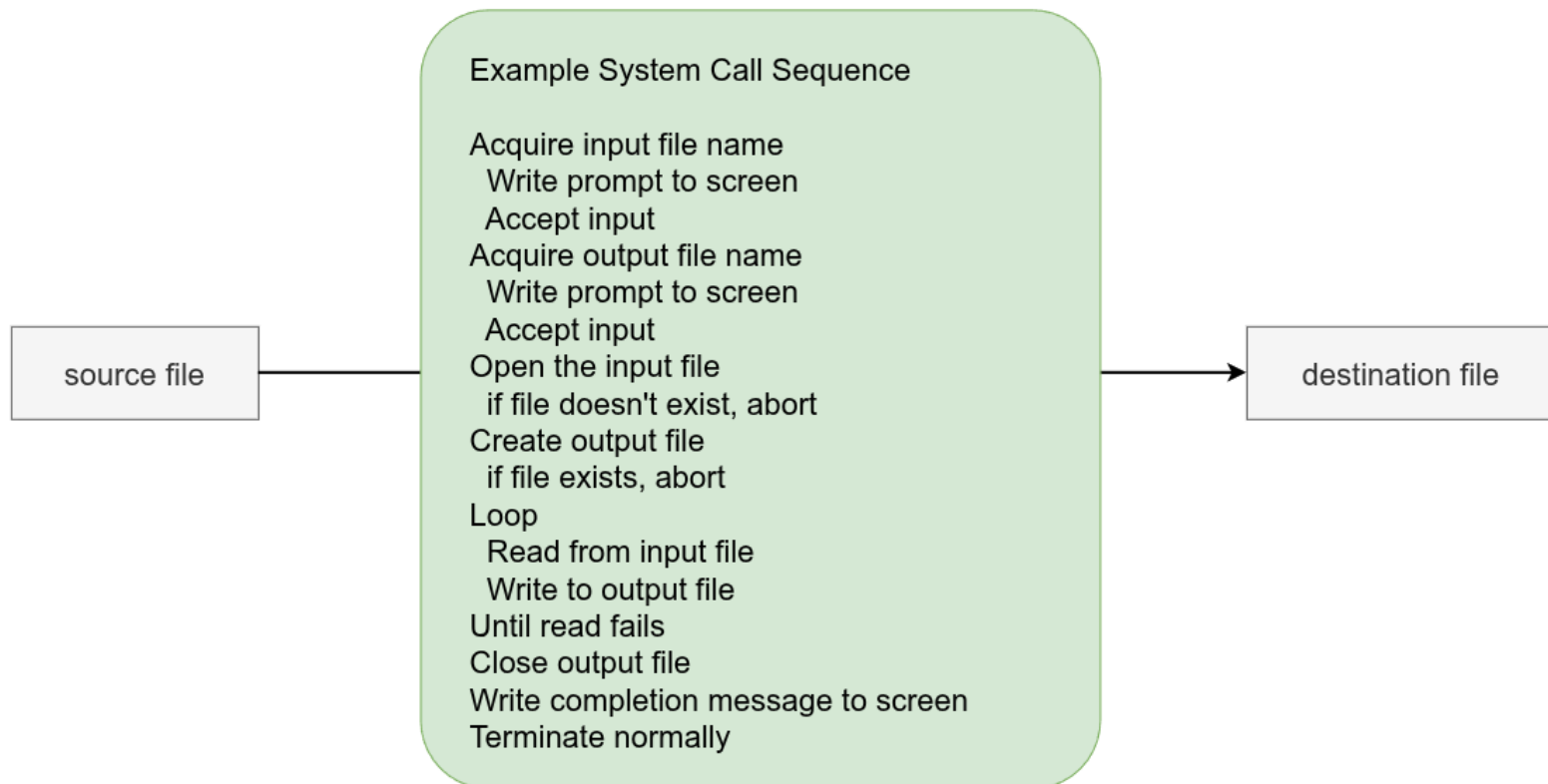
- programming interface provided by the OS to access its services
- typically written in an high-level language (C, C++, Rust)
- they are mainly accessed by programs through higher level

Application Programming Interface (API)

- several API standards:
 - Win32 for Windows
 - POSIX for POSIX-based systems (Linux, macOS, Solaris)
 - Java API for the Java Virtual Machine (JVM)

System Call - Example

- sequence of system calls to copy the content of a file in another one





Standard API - Example

```
#include <unistd.h>
```

```
ssize_t read(int fd, void *buf, size_t count)
```

Return type

name

parameters

➤ it permits to read the content of a file

- `int fd`: file to read
- `void *buf`: buffer where to save the read data
- `size_t count`: maximum data to be read



Example – open()

```
#include <fcntl.h>
```

```
...
```

```
int fd;
```

```
mode_t mode = S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH;
```

```
char *filename = "/tmp/file";
```

```
...
```

```
fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC,  
            mode);
```

```
...
```



Example - read()

```
#include <sys/types.h>
#include <unistd.h>

...
char buf[20];
size_t nbytes;
ssize_t bytes_read;
int fd;

...
nbytes = sizeof(buf);
bytes_read = read(fd, buf, nbytes);

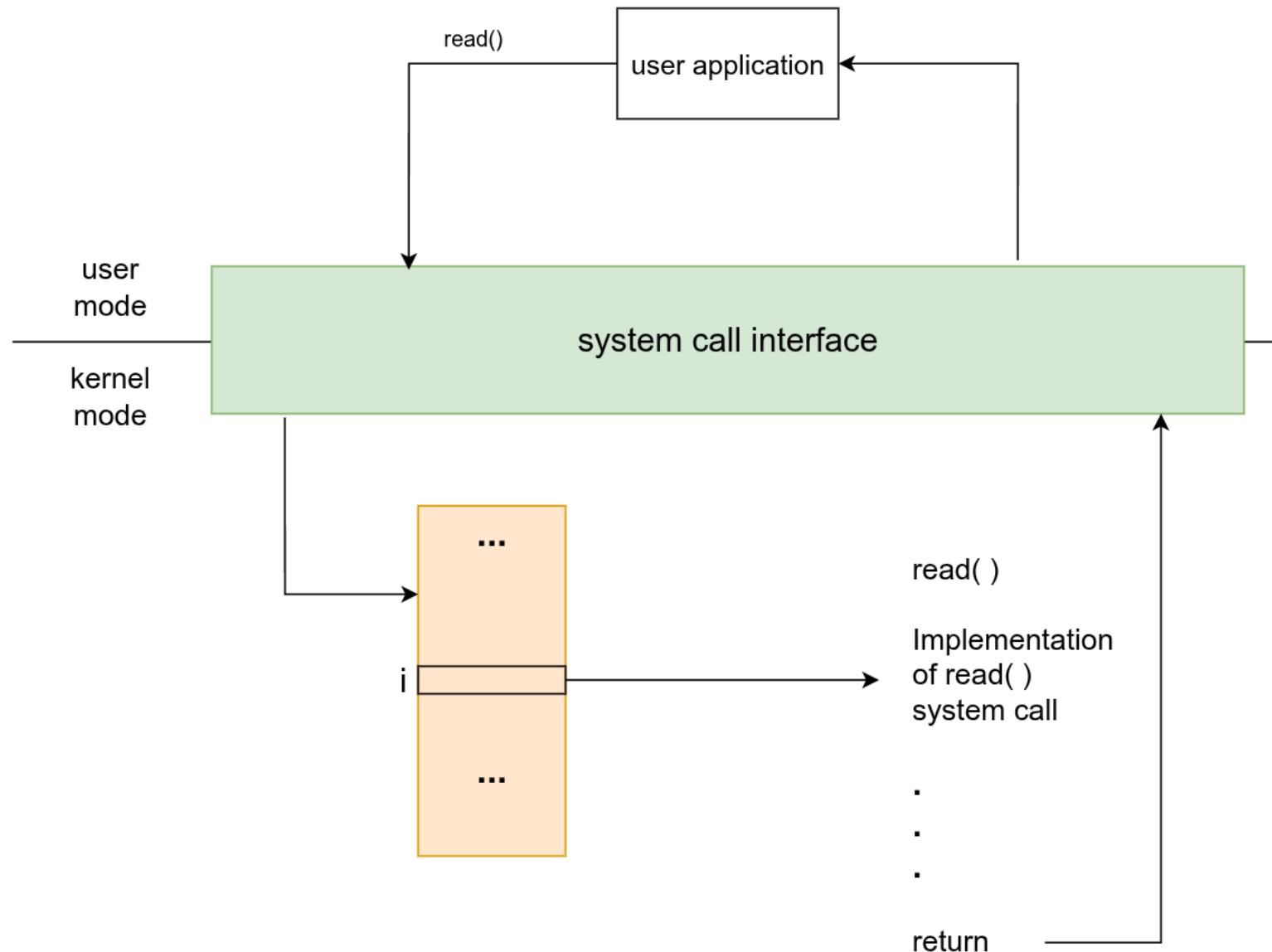
...
```



System Call Implementation

- each system call is typically associated with a number
 - the OS maintains a table following these numbers
- the system call interface calls the requested system call and returns the results
- the caller (user) does not need to know how the system calls are implemented, it just needs to follow the API rules

API Flow





System Call Types (I)

- Process Control
 - creation, termination
 - loading, execution
 - getting/setting process attributes
 - waiting for time or for event
 - allocating and freeing memory
 - debugging



System Call Types (II)

- management of files
 - creation, deletion
 - opening, closing
 - reading, writing
- management of devices
 - reading, writing
 - requesting, releasing
 - getting/setting device attributes



System Call Types (III)

- protection
 - access control to resources
 - getting and setting permissions
 - allowing and denying user access



Example – Standard C Library (I)

- in UNIX and Linux systems, the standard C library acts as a bridge between user applications and the operating system's kernel
- rather than requiring programs to communicate with the hardware directly, the library handles the complexities of the system-call interface.

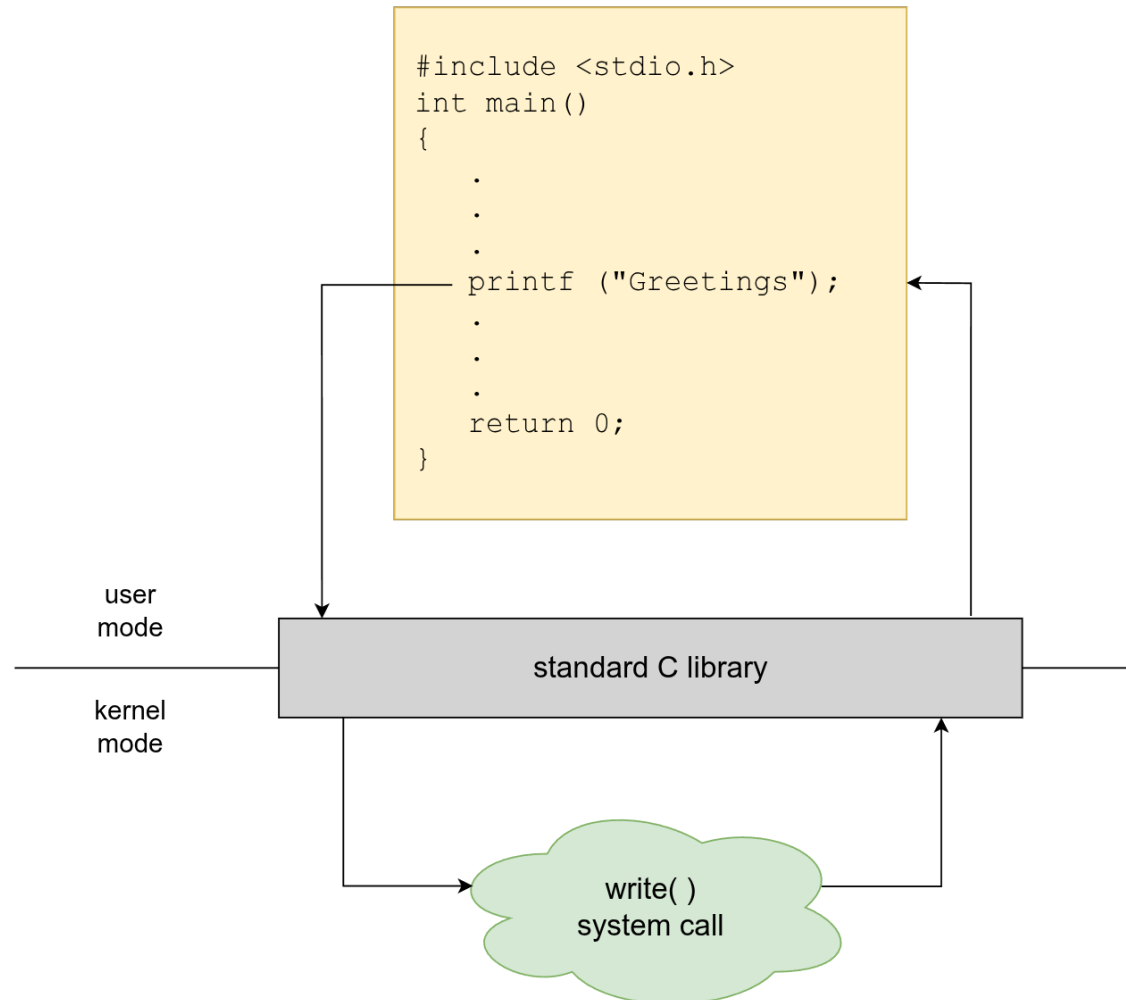


Example – Standard C Library (II)

- for example, when a C program executes a `printf()` statement:
 - the C library catches this request.
 - it translates the request into the appropriate system-level command, such as the `write()` system call.
 - the kernel processes the request and returns a result to the library.
 - the library then relays that return value back to the original user program.



Example – Standard C Library (III)





Policy vs Mechanism (I)

- OS abstracts implementation details
- user can describe the desired behavior
- **policy**: represents what needs to be done
- **mechanism**: represents how something needs to be done
- the mechanism is transparent to the policy



Policy vs Mechanism (II)

- typically the user does not want to implement, for example security, but wants to “state” what has to be behavior of the system
- the user defines its policy, defining a set of rules that the system needs to follow in order to enforce the user requirements
- the system is in charge of following the rules (policy) and implement the mechanism to follow them
- there can be different implementation the enforce the same rule